

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра Вычислительной техники**

**ОТЧЕТ**  
**по лабораторной работе №2**  
**по дисциплине «Нейронные сети»**  
**ТЕМА: РЕКУРРЕНТНЫЕ СЕТИ**

|                    |       |               |
|--------------------|-------|---------------|
| Студентка гр. 3311 | _____ | Суздалева А.  |
| Студент гр. 3311   | _____ | Шарпинский Д. |
| Студент гр. 3311   | _____ | Локтионов Т.  |
| Преподаватель      | _____ | Петруша П.Г.  |

**Санкт-Петербург**  
**2026**

## **Цель работы**

Целью лабораторной работы является получение практических навыков построения, обучения и сравнения рекуррентных нейронных сетей для решения задачи классификации временных последовательностей.

В рамках работы рассматриваются три архитектуры рекуррентных нейронных сетей: SimpleRNN, LSTM и GRU. Также исследуется влияние различных оптимизаторов и методов регуляризации на качество обучения модели. Для этого используются оптимизаторы SGD, Momentum, RMSProp и Adam, а также варианты без регуляризации, с L1-регуляризацией и с L2-регуляризацией.

В качестве прикладной задачи была выбрана классификация уровня нагрузки энергосистемы Испании на основе почасовых данных об энергопотреблении и погодных признаков. Модель должна по предыдущим 24 часам наблюдений предсказывать класс нагрузки на следующий час.

## **Задание работы**

- выбрать набор данных, представляющий собой временную последовательность;
- выполнить предварительный анализ и обработку данных;
- сформировать целевую переменную для задачи многоклассовой классификации;
- преобразовать данные во временные окна, пригодные для подачи в рекуррентную нейронную сеть;
- построить и обучить три архитектуры рекуррентных нейронных сетей: SimpleRNN, LSTM и GRU;
- проверить работу моделей с различными оптимизаторами: SGD, Momentum, RMSProp и Adam;
- проверить влияние регуляризации: без регуляризации, L1 и L2;
- сравнить полученные модели по значениям функции потерь и точности на тестовой выборке;
- определить лучшие и худшие конфигурации моделей.

## **Исходный набор данных**

В качестве исходного набора данных был рассмотрен датасет с данными энергопотребления в Испании:

Dataset:

<https://www.kaggle.com/datasets/nicholasjhana/energy-consumption-generation-prices-and-weather?resource=download>

Датасет представлен двумя файлами:

- energy\_dataset.csv — данные об энергосистеме Испании;
- weather\_features.csv — погодные данные по нескольким городам Испании.

Файл energy\_dataset.csv содержит почасовой временной ряд, включающий фактическую и прогнозную нагрузку энергосистемы, генерацию электроэнергии по различным источникам, прогнозные показатели и цены на электроэнергию. Размер исходной энергетической таблицы составляет 35064 строки и 29 столбцов, что соответствует примерно четырём годам почасовых наблюдений.

Файл weather\_features.csv содержит погодные признаки: температуру, минимальную и максимальную температуру, давление, влажность, скорость и направление ветра, осадки, снег и облачность. В отличие от энергетических данных, погодные данные представлены отдельно по нескольким городам Испании. Поэтому перед объединением с энергетическими данными их необходимо агрегировать по времени.

Задача была сформулирована следующим образом: по предыдущим 24 часам энергетических и погодных признаков предсказать класс нагрузки энергосистемы Испании на следующий час.

### **Первичный анализ данных**

На первом этапе были загружены два исходных файла:

```
energy_df = pd.read_csv("energy_dataset.csv")
```

```
weather_df = pd.read_csv("weather_features.csv")
```

После загрузки были просмотрены первые строки таблиц, размерность данных, список столбцов, типы данных, количество пропущенных значений и основные статистические характеристики.

Анализ показал, что энергетический датасет представляет собой основной временной ряд, где одна строка соответствует одному часу наблюдений. В нём присутствуют полностью пустые столбцы, константные столбцы, а также небольшое количество пропусков в отдельных числовых признаках.

Погодный датасет имеет больший размер, поскольку для каждого момента времени в нём хранится несколько строк — по разным городам. Также были обнаружены дубликаты по паре `dt_iso + city_name`, а в названии одного из городов присутствовал лишний пробел в начале строки. Кроме того, в погодных данных были обнаружены некорректные значения, например аномальное давление, равное 0 или 1008371.

### **Приведение временных признаков**

Для корректного объединения таблиц временные признаки были приведены к типу `datetime`:

```
energy["time"] = pd.to_datetime(energy["time"], utc=True)
weather["dt_iso"] = pd.to_datetime(weather["dt_iso"], utc=True)
```

Параметр `utc=True` использовался для корректной обработки часовых поясов. После этого обе таблицы были отсортированы по времени.

Проверка временных диапазонов показала, что оба набора данных относятся к одному и тому же периоду: с 2014-12-31 23:00:00 UTC по 2018-12-31 22:00:00 UTC. Это позволило объединять энергетические и погодные данные по временной метке без потери временной структуры.

В энергетическом датасете дубликатов по времени обнаружено не было. В погодных данных были обнаружены дубликаты по комбинации времени и города, поэтому перед объединением потребовалась дополнительная агрегация.

## Очистка энергетических данных

Из энергетического датасета были удалены полностью пустые столбцы и константные признаки. Полностью пустые признаки не содержат информации для модели, а константные признаки имеют одно и то же значение во всех строках, поэтому также не помогают при обучении.

После удаления неинформативных столбцов были обработаны пропущенные значения. Так как энергетические данные представлены в виде почасового временного ряда, пропуски были заполнены с использованием интерполяции по времени:

```
energy_clean = energy_clean.set_index("time")
energy_clean = energy_clean.interpolate(method="time")
energy_clean = energy_clean.ffill().bfill()
energy_clean = energy_clean.reset_index()
```

Интерполяция по времени позволяет восстановить отдельные пропущенные значения на основе соседних наблюдений. Если после интерполяции в начале или в конце таблицы оставались пропуски, они дополнительно заполнялись ближайшими доступными значениями с помощью прямого и обратного заполнения.

После выполнения этих операций в энергетическом датасете не осталось пропущенных значений, и данные стали пригодны для объединения с погодными признаками.

## Очистка погодных данных

Для погодного датасета сначала были очищены названия городов:

```
weather_clean["city_name"] = weather_clean["city_name"].str.strip()
```

Это позволило убрать лишние пробелы и корректно группировать данные по городам.

Для дальнейшей работы были выбраны только числовые погодные

признаки, имеющие физический смысл:

```
weather_feature_cols = [  
    "temp",  
    "temp_min",  
    "temp_max",  
    "pressure",  
    "humidity",  
    "wind_speed",  
    "wind_deg",  
    "rain_1h",  
    "rain_3h",  
    "snow_3h",  
    "clouds_all"  
]
```

Затем были обработаны некорректные погодные значения. Давление вне диапазона 900–1100 hPa было заменено на пропуски, так как такие значения являются физически неправдоподобными для атмосферного давления. Также были обработаны некорректные значения влажности и подозрительно высокая скорость ветра.

Температурные признаки были переведены из Кельвинов в градусы Цельсия:

```
for col in ["temp", "temp_min", "temp_max"]:  
    weather_clean[col] = weather_clean[col] - 273.15
```

Этот шаг не является обязательным для нейронной сети, но делает признаки более интерпретируемыми при анализе данных.

### **Агрегация погодных данных**

Так как погодные данные были представлены отдельно по городам, их нельзя было напрямую объединить с энергетическим датасетом. Для решения этой проблемы была выполнена двухэтапная агрегация.

Сначала были агрегированы дубликаты по одному городу и одному часу:

```
weather_by_city_hour = (  
    weather_clean  
        .groupby(["dt_iso", "city_name"],  
as_index=False)[weather_feature_cols]  
        .mean()  
    )
```

После этого погодные признаки были усреднены по всем городам для каждого часа:

```
weather_agg = (  
    weather_by_city_hour  
        .groupby("dt_iso", as_index=False)[weather_feature_cols]  
        .mean()  
    )
```

Таким образом, для каждого часа была получена одна строка с усреднёнными погодными параметрами по городам Испании.

После агрегации возможные пропуски, возникшие из-за очистки выбросов, были заполнены временной интерполяцией, а затем методами прямого и обратного заполнения:

```
weather_agg = weather_agg.set_index("dt_iso")  
weather_agg = weather_agg.interpolate(method="time").ffill().bfill()  
weather_agg = weather_agg.reset_index()
```

Затем колонка `dt_iso` была переименована в `time`, чтобы её можно было использовать для объединения с энергетическими данными.

### **Объединение энергетических и погодных данных**

После очистки и агрегации два датасета были объединены по временной колонке:

```
df = pd.merge(
```

```
energy_clean,  
weather_agg,  
on="time",  
how="inner"  
)
```

В результате был получен итоговый датасет размером 35064 строки и 32 столбца. Диапазон времени сохранился полностью: с 2014-12-31 23:00:00 UTC по 2018-12-31 22:00:00 UTC.

После объединения пропущенных значений не осталось. Это означает, что энергетические и погодные данные были успешно приведены к единому почасовому временному ряду.

Итоговый датасет содержит признаки энергосистемы Испании, включая генерацию по источникам, прогнозы, фактическую нагрузку и цены на электроэнергию, а также агрегированные погодные признаки: температуру, давление, влажность, ветер, осадки и облачность.

### **Формирование целевой переменной**

В качестве основы для целевой переменной был использован столбец `total load actual`, который содержит фактическую нагрузку энергосистемы.

Анализ показал, что значения фактической нагрузки изменяются примерно в диапазоне от 18041 до 41015. Среднее значение составляет около 28698, медиана — около 28902.

Так как в лабораторной работе требуется решить задачу многоклассовой классификации, непрерывное значение нагрузки было преобразовано в три класса:

- 0 — низкая нагрузка;
- 1 — средняя нагрузка;
- 2 — высокая нагрузка.

Границы классов были выбраны по квантилям 33% и 66%:



```
q1 = df[target_col].quantile(0.33)
q2 = df[target_col].quantile(0.66)
```

В результате были получены следующие пороги:

- низкая нагрузка — значения до 25972;
- средняя нагрузка — значения от 25972 до 31058;
- высокая нагрузка — значения выше 31058.

Такой способ разбиения позволил получить почти сбалансированное распределение классов:

- низкая нагрузка — 11575 наблюдений;
- средняя нагрузка — 11568 наблюдений;
- высокая нагрузка — 11921 наблюдение.

Это важно для обучения модели, поскольку при близком количестве объектов в каждом классе модель не будет смещаться в сторону одного доминирующего класса.

### **Добавление временных признаков**

После формирования целевой переменной в датасет были добавлены календарные признаки:

```
df["hour"] = df["time"].dt.hour
df["day_of_week"] = df["time"].dt.dayofweek
df["month"] = df["time"].dt.month
```

Эти признаки позволяют модели учитывать суточную, недельную и сезонную периодичность. Для задачи прогнозирования энергопотребления это важно, так как нагрузка зависит от времени суток, дня недели и месяца.

После этого в качестве входных признаков были выбраны все числовые столбцы, кроме целевой переменной `load_class`. Входные данные включали как энергетические признаки, так и погодные и календарные признаки.

Количество входных признаков после подготовки составило 34.

## Нормализация признаков

Перед обучением нейронных сетей входные признаки были нормализованы с помощью `StandardScaler`:

```
scaler = StandardScaler()
scaler.fit(df_model.loc[:train_size - 1, feature_cols])
features_scaled = scaler.transform(df_model[feature_cols])
```

Нормализация необходима, поскольку признаки имеют разные масштабы. Например, нагрузка измеряется десятками тысяч, температура — десятками градусов, влажность — значениями от 0 до 100, а календарные признаки имеют ещё меньшие диапазоны.

Важно, что `StandardScaler` обучался только на обучающей части данных. Это сделано для предотвращения утечки информации из тестовой выборки. После обучения `scaler` применялся ко всему набору данных.

## Разделение на обучающую и тестовую выборки

Так как данные являются временным рядом, разделение выполнялось не случайно, а в хронологическом порядке. Первые 80% данных использовались для обучения, оставшиеся 20% — для тестирования:

```
train_size = int(len(df_model) * 0.8)
```

Такой подход соответствует реальной задаче прогнозирования, где модель обучается на прошлых данных и проверяется на более поздних наблюдениях.

## Формирование временных окон

Для подачи данных в рекуррентную нейронную сеть исходный временной ряд был преобразован во временные окна.

Размер окна был выбран равным 24:

```
window_size = 24
```

Это означает, что каждый обучающий пример содержит 24 предыдущих часа наблюдений. По этим 24 часам модель должна предсказать класс нагрузки

на следующий, 25-й час.

Формирование последовательностей выполнялось функцией `create_sequences`. Для каждого окна формировались:

- $X[i]$  — матрица размера  $24 \times 34$ , содержащая 24 часа истории и 34 признака на каждом шаге;
- $y[i]$  — класс нагрузки следующего часа.

Итоговая форма входных данных:

$X_{train}$ : (28027, 24, 34)  
 $X_{test}$ : (7013, 24, 34)

Таким образом, обучающая выборка содержит 28027 временных последовательностей, а тестовая — 7013 последовательностей. Каждая последовательность состоит из 24 временных шагов и 34 признаков.

Целевая переменная была преобразована в one-hot encoding, так как для обучения использовалась функция потерь `categorical_crossentropy`:

```
y_train_cat = to_categorical(y_train, num_classes=num_classes)
y_test_cat = to_categorical(y_test, num_classes=num_classes)
```

### Построение модели SimpleRNN

Первой была построена простая рекуррентная нейронная сеть SimpleRNN.

Архитектура модели имеет следующий вид:

```
Input(shape=(24, 34))
SimpleRNN(64, activation="tanh")
Dropout(0.2)
Dense(32, activation="relu")
Dense(3, activation="softmax")
```

Слой SimpleRNN обрабатывает последовательность из 24 временных шагов и формирует скрытое состояние, в котором сохраняется информация о предыдущих элементах последовательности. Затем это состояние передаётся

в полносвязный слой Dense, после чего выходной слой с функцией активации softmax выдаёт вероятности трёх классов нагрузки.

Dropout с коэффициентом 0.2 используется для снижения риска переобучения.

Для SimpleRNN были проверены все комбинации оптимизаторов и регуляризации:

- оптимизаторы: SGD, Momentum, RMSProp, Adam;
- регуляризация: отсутствует, L1, L2.

Всего для SimpleRNN было обучено 12 конфигураций.

### **Построение модели LSTM**

Второй архитектурой была модель LSTM.

LSTM является развитием обычной RNN и лучше подходит для работы с временными зависимостями, поскольку использует механизм ячеек памяти и управляющих вентилей. Благодаря этому LSTM может сохранять информацию на более длинных временных промежутках и частично решает проблему затухающих градиентов.

Архитектура модели LSTM:

```
Input(shape=(24, 34))  
LSTM(64, activation="tanh")  
Dropout(0.2)  
Dense(32, activation="relu")  
Dense(3, activation="softmax")
```

Как и в случае с SimpleRNN, модель завершалась полносвязным слоем и выходным слоем softmax, который выдаёт вероятности трёх классов: низкой, средней и высокой нагрузки.

Для LSTM также были проверены 12 конфигураций: четыре оптимизатора и три варианта регуляризации.

### **Построение модели GRU**

Третьей архитектурой была GRU.

GRU является упрощённой альтернативой LSTM. Она также использует управляющие механизмы для сохранения и обновления информации, но имеет более простую структуру. За счёт этого GRU обычно обучается быстрее, чем LSTM, при сопоставимом качестве на многих задачах временных рядов.

Архитектура модели GRU:

```
Input(shape=(24, 34))  
GRU(64, activation="tanh")  
Dropout(0.2)  
Dense(32, activation="relu")  
Dense(3, activation="softmax")
```

Для GRU также были протестированы все комбинации оптимизаторов и регуляризации. Всего было обучено 12 конфигураций.

### **Используемые оптимизаторы**

В работе были протестированы четыре оптимизатора.

Обычный SGD использовался со скоростью обучения 0.01:

```
SGD(learning_rate=0.01)
```

Momentum был реализован как SGD с параметром momentum=0.9:

```
SGD(learning_rate=0.01, momentum=0.9)
```

RMSProp использовался со скоростью обучения 0.001:

```
RMSprop(learning_rate=0.001)
```

Adam также использовался со скоростью обучения 0.001:

```
Adam(learning_rate=0.001)
```

SGD является базовым методом градиентной оптимизации. Momentum улучшает SGD за счёт учёта предыдущих направлений движения по градиенту. RMSProp адаптирует скорость обучения для разных параметров модели. Adam сочетает идеи Momentum и адаптивного изменения скорости

обучения, поэтому часто показывает стабильные результаты при обучении нейронных сетей.

### **Регуляризация**

Для каждой архитектуры были проверены три варианта:

- без регуляризации;
- L1-регуляризация;
- L2-регуляризация.

Коэффициент регуляризации был задан равным 0.001.

Регуляризация применялась к весам рекуррентных слоёв и полносвязных слоёв:

```
kernel_regularizer=reg  
recurrent_regularizer=reg  
bias_regularizer=reg
```

L1-регуляризация штрафует модель за абсолютные значения весов и может приводить к занулению части коэффициентов. L2-регуляризация штрафует модель за квадраты весов и обычно используется для снижения переобучения за счёт ограничения слишком больших весовых коэффициентов.

### **Обучение моделей**

Все модели обучались по единой схеме.

Для каждой комбинации архитектуры, оптимизатора и регуляризации выполнялось обучение модели с одними и теми же параметрами:

```
epochs = 15  
batch_size = 64  
validation_split = 0.2
```

Для контроля переобучения использовался EarlyStopping:

```
EarlyStopping(  
    monitor="val_loss",
```

```
patience=3,  
restore_best_weights=True  
)
```

Это означает, что обучение останавливалось, если значение ошибки на валидационной выборке не улучшалось в течение трёх эпох. После остановки восстанавливались лучшие веса модели.

Для воспроизводимости перед обучением каждой модели задавались фиксированные значения генераторов случайных чисел:

```
np.random.seed(42)  
tf.random.set_seed(42)
```

После обучения каждая модель оценивалась на тестовой выборке:

```
test_loss, test_accuracy = model.evaluate(  
X_test,  
y_test_cat,  
verbose=0  
)
```

Для каждой конфигурации сохранялись следующие значения:

- архитектура модели;
- оптимизатор;
- тип регуляризации;
- значение test\_loss;
- значение test\_accuracy;
- количество реально пройденных эпох.

### **Итоговая логика работы модели**

Вся задача сводится к следующей последовательности действий.

Сначала исходные данные об энергопотреблении и погоде приводятся к единому почасовому временному ряду. Затем фактическая нагрузка total load

actual преобразуется в категориальный признак `load_class`, принимающий три значения: низкая, средняя и высокая нагрузка.

После этого из временного ряда формируются окна длиной 24 часа. Каждое окно представляет собой матрицу  $24 \times 34$ : 24 временных шага и 34 признака на каждом шаге. На выходе модель должна предсказать класс нагрузки для следующего часа.

Рекуррентная сеть последовательно обрабатывает 24 входных вектора, формирует внутреннее состояние и передаёт его в полносвязные слои. Последний слой с функцией активации `softmax` выдаёт три вероятности, соответствующие трём классам нагрузки. Предсказанным классом считается класс с наибольшей вероятностью.

### Сравнение моделей

После обучения всех моделей было выполнено сравнение полученных конфигураций по двум основным метрикам: `test_accuracy` и `test_loss`. Для каждой архитектуры рассматривались 12 вариантов: 4 оптимизатора и 3 режима регуляризации. Всего было обучено и оценено 36 моделей.

Для анализа результатов отдельно сравнивались модели SimpleRNN, LSTM и GRU. Также для каждой архитектуры были построены графики изменения точности и функции потерь на обучающей и валидационной выборках для лучшей и худшей конфигурации.

### Результаты SimpleRNN

Для архитектуры SimpleRNN лучшей конфигурацией по точности на тестовой выборке стала модель с оптимизатором `RMSProp` и `L1`-регуляризацией. Она показала значение `test_accuracy = 0.9331`. Это является наибольшей точностью не только среди моделей SimpleRNN, но и среди всех рассмотренных конфигураций в работе.

На графиках обучения видно, что модель `SimpleRNN_rmsprop_l1` достаточно быстро достигает высокой точности. Уже после первых эпох значение `accuracy` заметно возрастает, после чего рост становится более



плавным. Значения accuracy на обучающей и валидационной выборках находятся близко друг к другу, что говорит об отсутствии сильного переобучения. Функция потерь также стабильно уменьшается как на обучающей, так и на валидационной выборке.

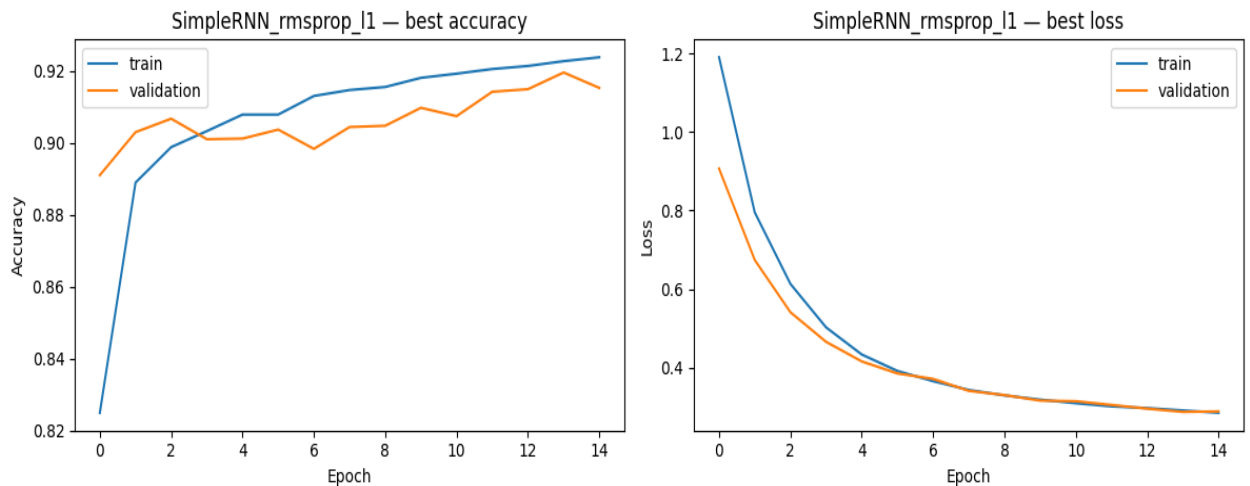


Рис. 1 — Графики accuracy и loss для лучшей модели SimpleRNN (rmsprop + l1)

Худшей конфигурацией SimpleRNN по точности стала модель с обычным SGD без регуляризации. Она показала test\_accuracy = 0.9069. При этом даже худшая конфигурация SimpleRNN дала достаточно высокую точность, что говорит о том, что задача хорошо решается даже простой рекуррентной архитектурой. Однако по сравнению с лучшей моделью с RMSProp качество ниже, а обучение происходит менее эффективно.

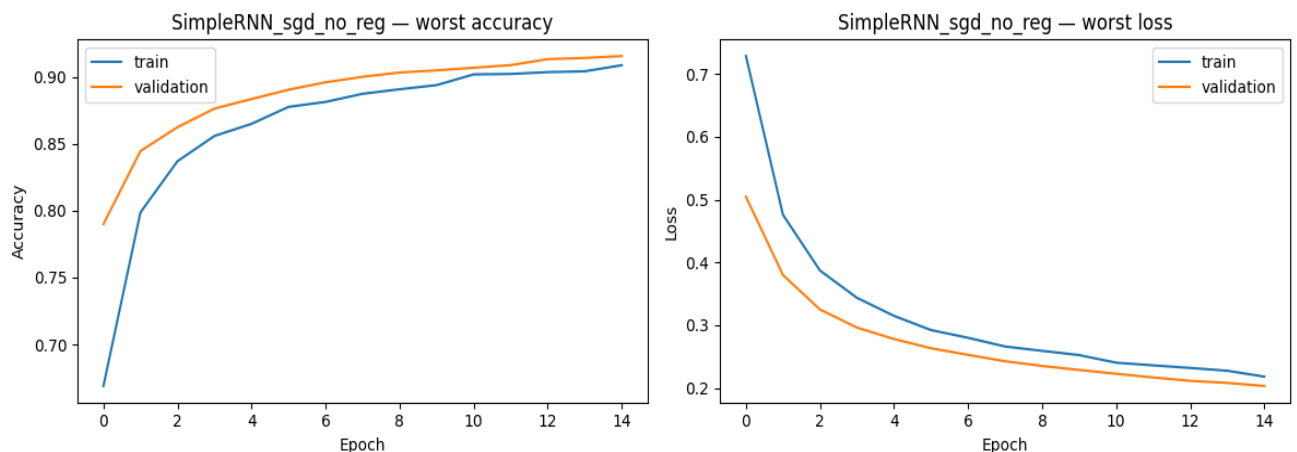


Рис. 2 — Графики accuracy и loss для худшей модели SimpleRNN (sgd + no\_reg)

По итоговому сравнению всех конфигураций SimpleRNN лучшие значения accuracy показали варианты с оптимизатором RMSProp и регуляризацией L1 или L2. Значения accuracy для лучших моделей составили:

- rmsprop + l1 — 0.9331;
- rmsprop + l2 — 0.9323;
- adam + l2 — 0.9287.

Наихудшее значение accuracy получила конфигурация `sgd + none` — 0.9069. При этом по значению `test_loss` худшей оказалась конфигурация `sgd + l1`, у которой loss составил 0.7996. Это показывает, что худшая модель по accuracy и худшая модель по loss не всегда совпадают.

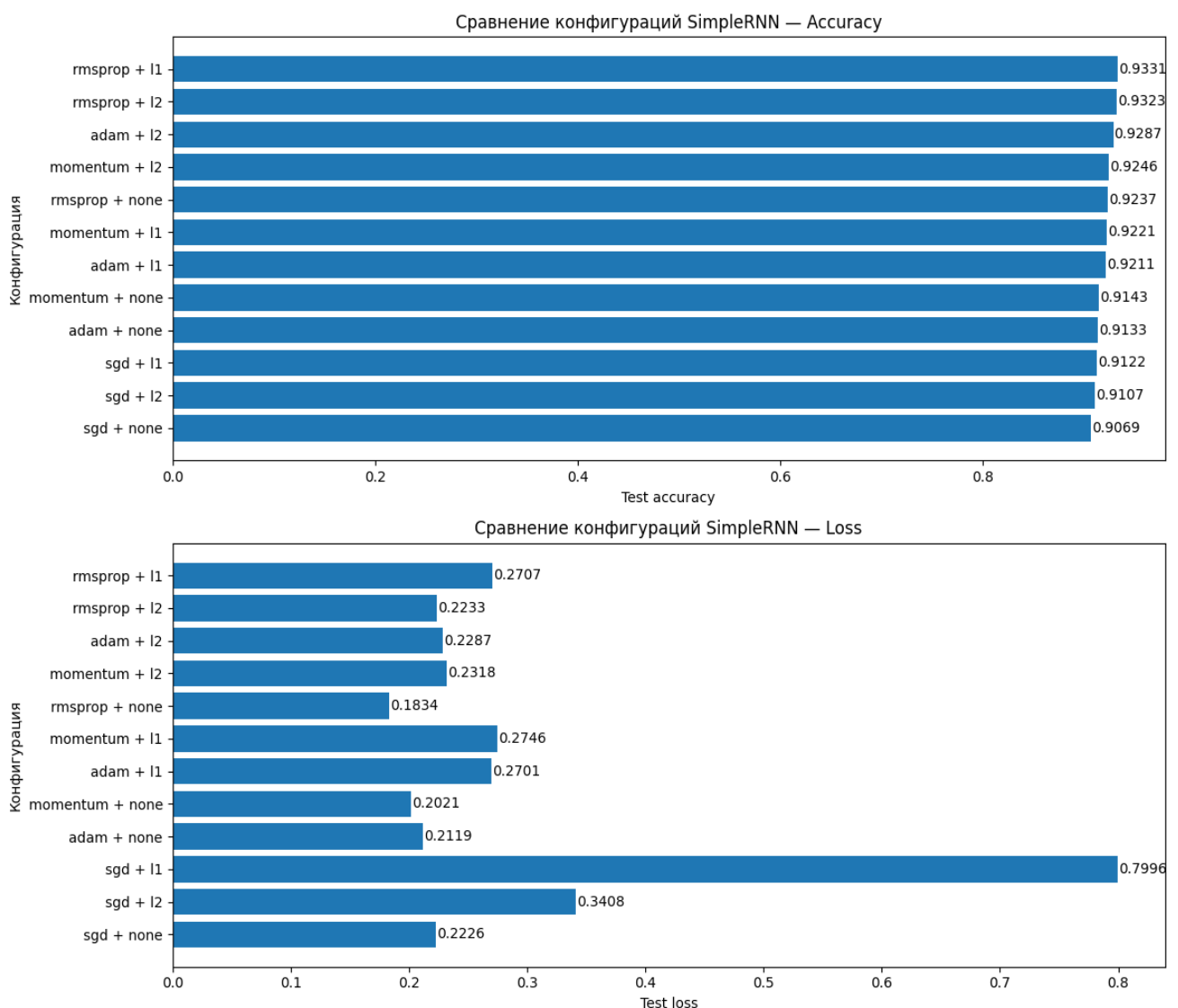


Рис. 3 — Сравнение конфигураций SimpleRNN по test accuracy и test loss

Таким образом, для SimpleRNN наилучший результат по точности был

получен при использовании RMSProp и L1-регуляризации. При этом обычный SGD оказался самым слабым вариантом оптимизации.

## Результаты LSTM

Для архитектуры LSTM лучшей конфигурацией по точности стала модель с оптимизатором Adam и L2-регуляризацией. Она показала  $\text{test\_accuracy} = 0.9280$  и  $\text{test\_loss} = 0.2386$ .

На графиках обучения видно, что модель LSTM\_adam\_l2 обучалась стабильно. Точность на обучающей и валидационной выборках постепенно увеличивалась и к концу обучения достигла примерно одинакового уровня. Функция потерь также плавно снижалась. Близость графиков train и validation указывает на то, что модель не демонстрирует выраженного переобучения.

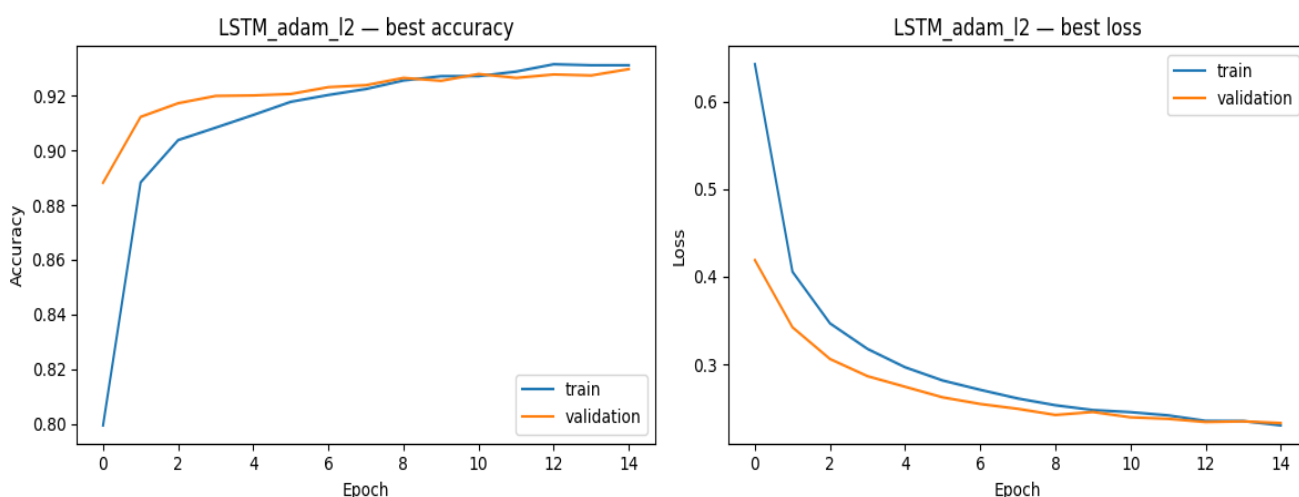


Рис. 4 — Графики accuracy и loss для лучшей модели LSTM (adam + l2)

Худшей конфигурацией LSTM стала модель с оптимизатором SGD и L1-регуляризацией. Она показала  $\text{test\_accuracy} = 0.8798$  и  $\text{test\_loss} = 1.0773$ . Это худший результат среди всех моделей LSTM и один из худших результатов во всей работе.

На графиках видно, что accuracy у данной модели растёт, однако остаётся заметно ниже, чем у лучших конфигураций. Значение loss остаётся высоким на протяжении всего обучения. Это говорит о том, что сочетание обычного SGD и L1-регуляризации для данной задачи оказалось неудачным: модель обучалась медленнее и хуже приближала целевую зависимость.

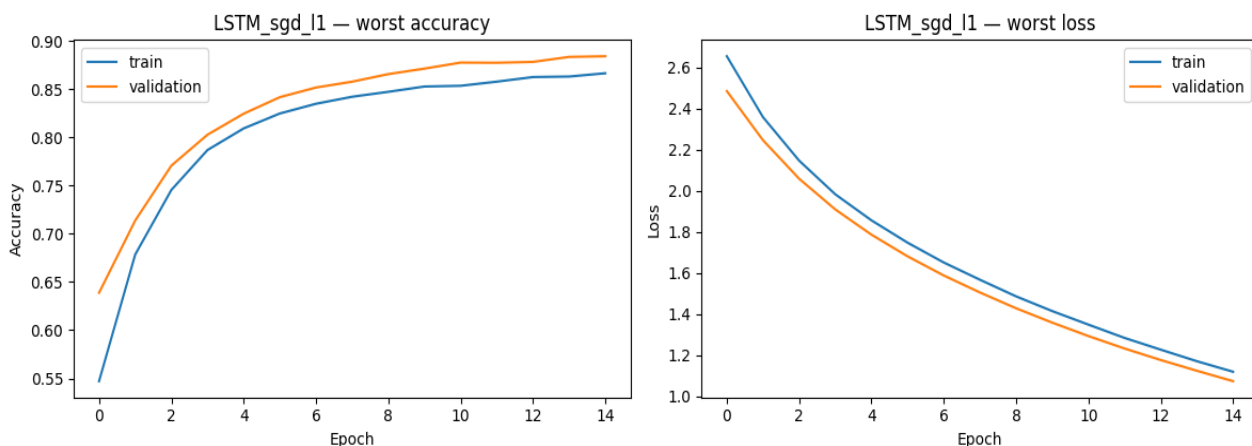


Рис. 5 — Графики ассурасу и loss для худшей модели LSTM (sgd + l1)

В сравнении всех конфигураций LSTM лучшие значения ассурасу показали следующие варианты:

- adam + l2 — 0.9280;
- rmsprop + l1 — 0.9260;
- rmsprop + l2 — 0.9259;
- adam + l1 — 0.9219.

Наихудшие результаты показали конфигурации с обычным SGD:

- sgd + l2 — 0.8923;
- sgd + none — 0.8821;
- sgd + l1 — 0.8798.

По значению loss минимальные результаты среди LSTM получили конфигурации без регуляризации с Momentum, RMSProp и Adam. Однако по ассурасу лучшей оказалась именно модель adam + l2. Это показывает, что при сравнении моделей важно учитывать обе метрики: точность классификации и значение функции потерь.

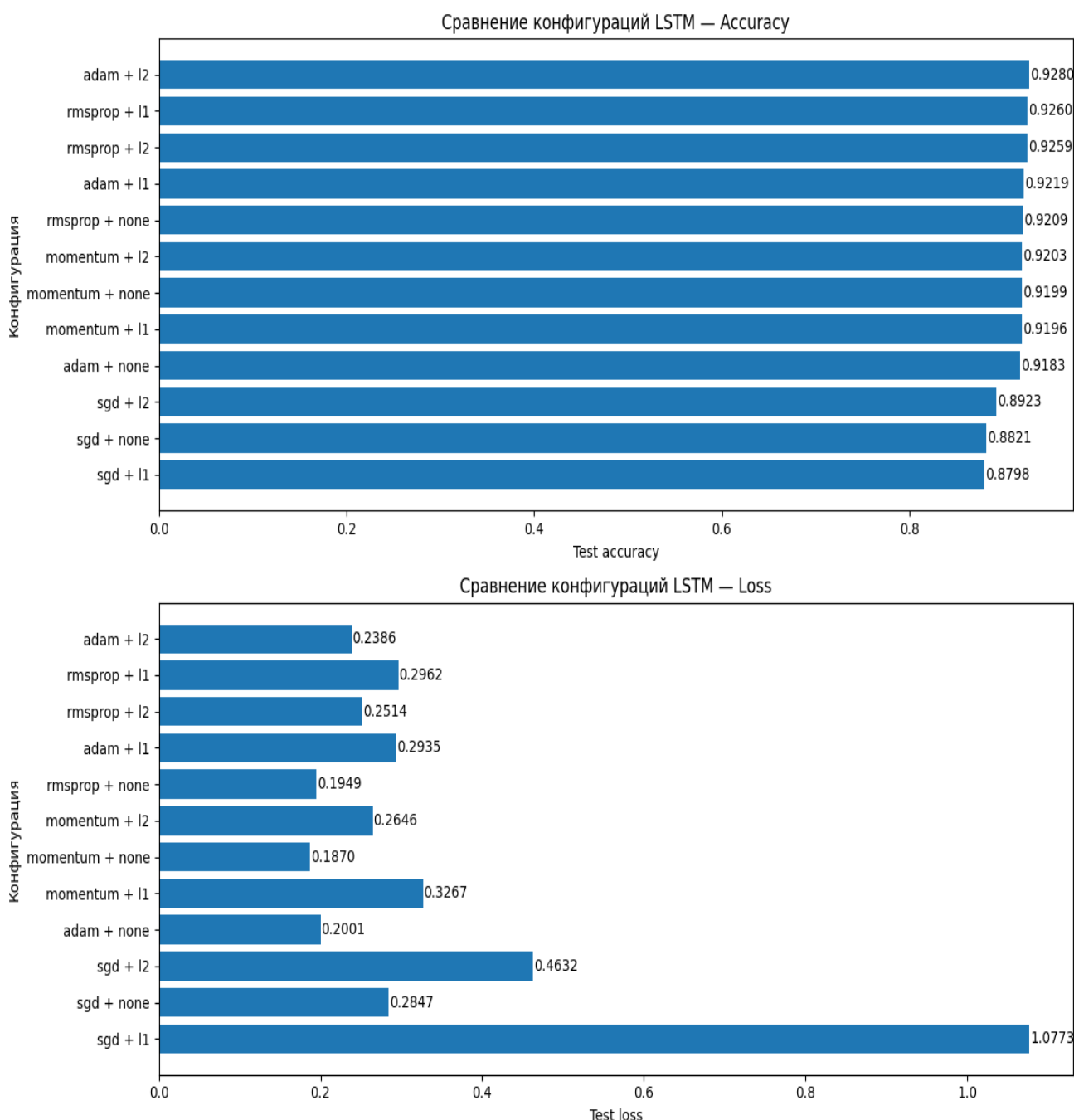


Рис. 6 — Сравнение конфигураций LSTM по test accuracy и test loss

Таким образом, для LSTM наиболее удачным вариантом стала комбинация Adam и L2-регуляризации. Обычный SGD, особенно в сочетании с L1-регуляризацией, показал худшее качество.

### Результаты GRU

Для архитектуры GRU лучшей конфигурацией стала модель с оптимизатором Adam без регуляризации. Она показала  $\text{test\_accuracy} = 0.9287$  и  $\text{test\_loss} = 0.1740$ . По accuracy такая же точность была у варианта adam + l2, однако у модели без регуляризации значение loss оказалось заметно ниже.

Поэтому в качестве лучшей GRU-модели была выбрана конфигурация GRU\_adam\_no\_reg.

На графиках обучения видно, что модель быстро достигает высокой точности. Ассурасу на обучающей выборке продолжает расти до конца обучения, а validation ассурасу стабилизируется примерно на уровне 0.93. При этом validation loss сначала снижается, а затем выходит на плато и немного колеблется. Это может указывать на лёгкое переобучение к концу обучения, однако итоговое качество на тестовой выборке остаётся высоким.

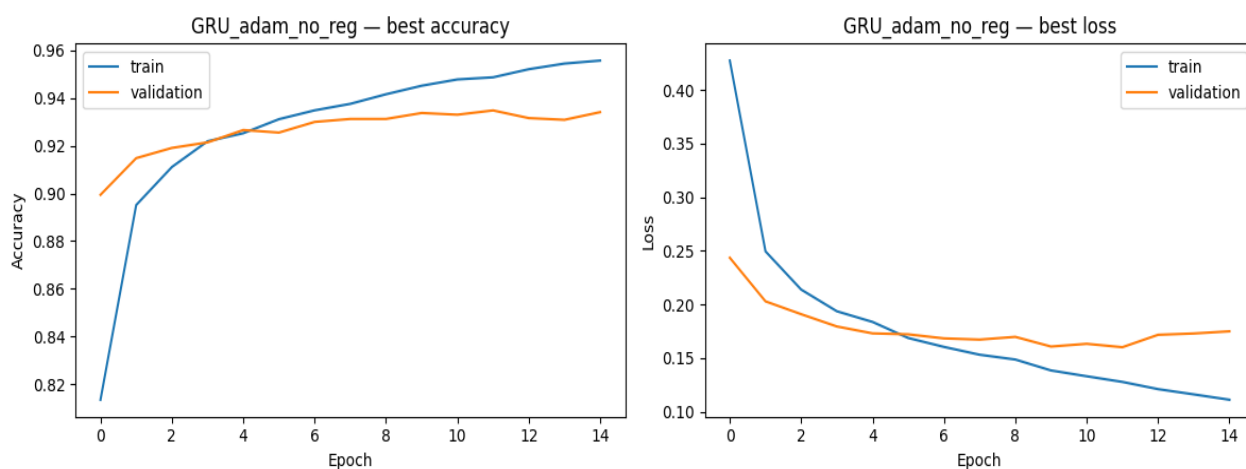


Рис. 7 — Графики ассурасу и loss для лучшей модели GRU (adam + no\_reg)

Худшей конфигурацией GRU стала модель с оптимизатором SGD и L1-регуляризацией. Она показала test\_accuracy = 0.8952 и test\_loss = 0.9556. Как и в случае с LSTM, сочетание обычного SGD и L1-регуляризации оказалось неэффективным.

На графиках видно, что модель постепенно обучается, ассурасу растёт, а loss уменьшается. Однако итоговые значения всё равно значительно хуже, чем у конфигураций с Adam, RMSProp и Momentum. Это говорит о том, что при выбранных параметрах обучения SGD хуже подходит для данной задачи.

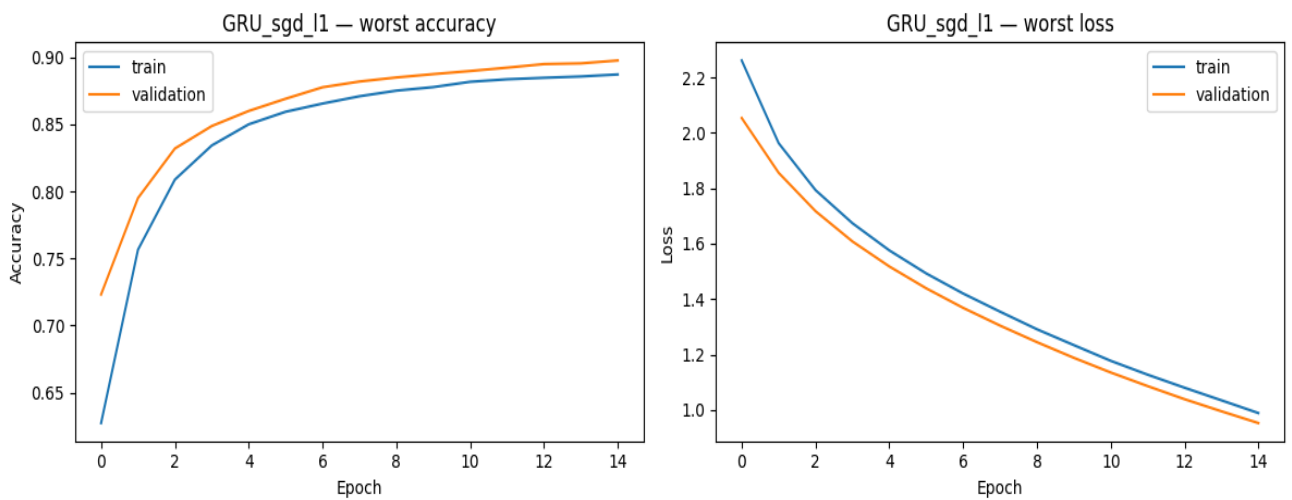


Рис. 8 — Графики accuracy и loss для худшей модели GRU (sgd + l1)

Сравнение всех конфигураций GRU показало, что лучшие результаты по accuracy получили:

- adam + none — 0.9287;
- adam + l2 — 0.9287;
- rmsprop + l2 — 0.9269;
- rmsprop + none — 0.9269.

Наихудшие результаты получили конфигурации с SGD:

- sgd + none — 0.9039;
- sgd + l2 — 0.9018;
- sgd + l1 — 0.8952.

По значению функции потерь лучшей стала конфигурация adam + none с test\_loss = 0.1740. Это также минимальное значение loss среди всех моделей, рассмотренных в работе.

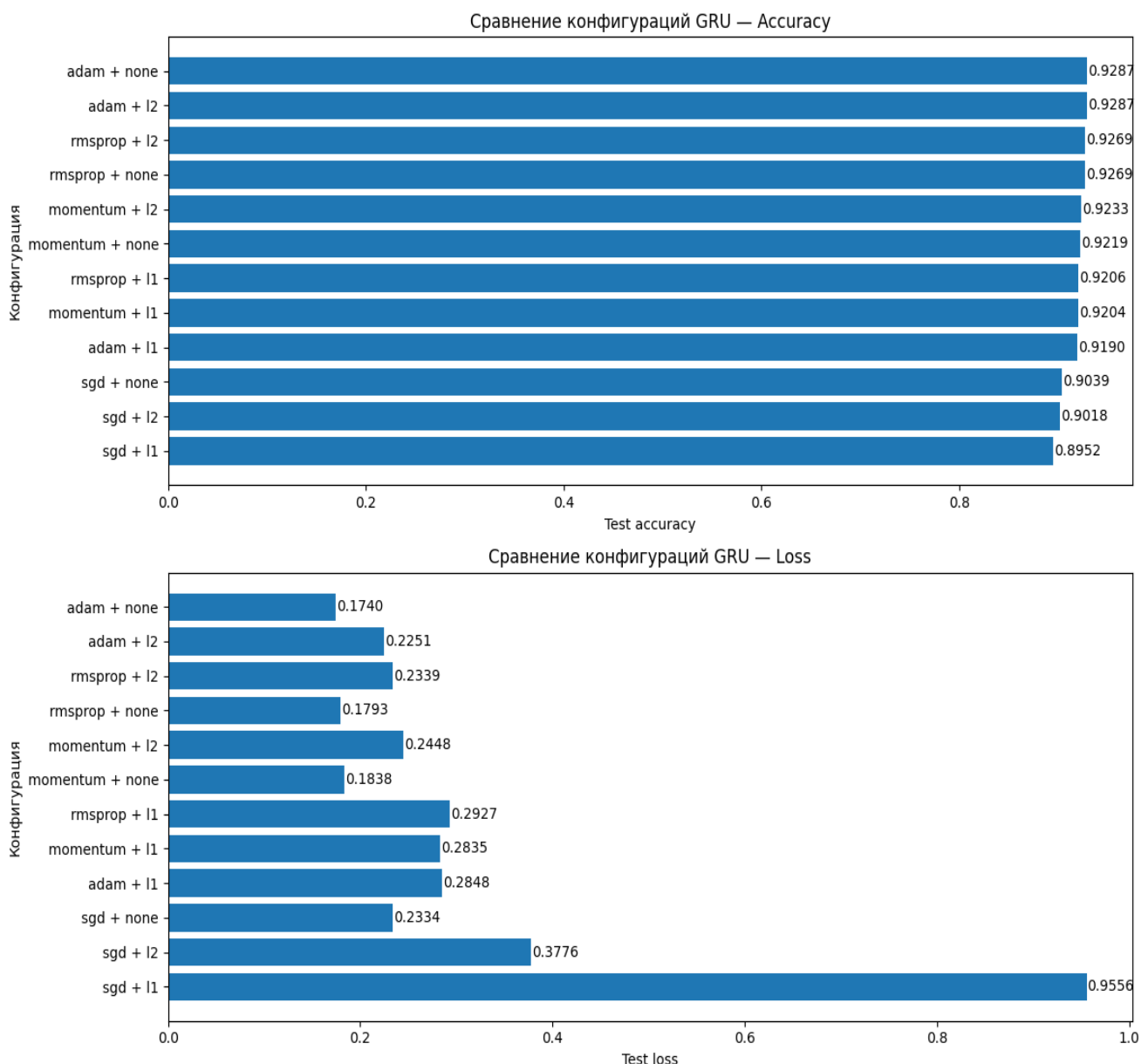


Рис. 9 — Сравнение конфигураций GRU по test accuracy и test loss

Таким образом, для GRU лучшей оказалась модель с Adam без регуляризации. Она уступила лучшей SimpleRNN по accuracy, но показала лучшее значение test loss среди всех моделей.

### Общее сравнение архитектур

Наибольшую точность на тестовой выборке показала модель SimpleRNN\_rmsprop\_l1 — 0.9331. Это делает её лучшей моделью по основной метрике классификации.

При этом минимальное значение функции потерь получила модель GRU\_adam\_no\_reg — 0.1740. Это означает, что GRU-модель формировала более уверенные и стабильные вероятностные предсказания, хотя по accuracy



немного уступила лучшей SimpleRNN.

Модель LSTM также показала высокий результат, однако её лучшая конфигурация оказалась немного слабее SimpleRNN и GRU по accuracy. Это может быть связано с тем, что длина входной последовательности составляла 24 часа, то есть зависимость была не слишком длинной. Для такой задачи простая рекуррентная сеть могла оказаться достаточной, тогда как более сложные архитектуры LSTM и GRU не дали существенного преимущества по точности.

Общий анализ оптимизаторов показал, что Adam и RMSProp чаще давали лучшие результаты, чем обычный SGD. Momentum также показывал приемлемое качество, но в большинстве случаев уступал Adam и RMSProp. Обычный SGD стабильно оказывался среди худших конфигураций, особенно при использовании L1-регуляризации.

Также можно отметить, что регуляризация не всегда улучшала качество. Для LSTM лучшей стала модель с L2-регуляризацией, для SimpleRNN — с L1-регуляризацией, а для GRU лучший результат по loss был получен вообще без регуляризации. Следовательно, влияние регуляризации зависит от конкретной архитектуры и оптимизатора.

## **Вывод**

В ходе выполнения лабораторной работы была решена задача многоклассовой классификации временного ряда на основе данных энергопотребления и погодных условий в Испании.

На этапе подготовки данных были загружены два исходных файла: энергетический датасет и погодный датасет. Энергетические данные представляли собой почасовой временной ряд, а погодные данные были представлены по нескольким городам. Для корректного объединения данные были приведены к единому формату времени, очищены от пропусков, дубликатов и некорректных значений.

Из энергетического датасета были удалены полностью пустые и константные признаки. Пропущенные значения были заполнены

интерполяцией по времени. В погодных данных были очищены названия городов, обработаны выбросы в давлении, влажности и скорости ветра, а температура была переведена из Кельвинов в градусы Цельсия. После этого погодные признаки были агрегированы по каждому часу путём усреднения по городам.

В результате объединения был получен единый датасет размером 35064 строки и 32 столбца без пропущенных значений. На основе столбца `total load actual` была сформирована целевая переменная `load_class`, содержащая три класса нагрузки: низкую, среднюю и высокую. Разбиение по квантилям позволило получить почти сбалансированное распределение классов.

Для обучения рекуррентных нейронных сетей данные были преобразованы во временные окна длиной 24 часа. Каждый входной объект представлял собой последовательность из 24 временных шагов и 34 признаков. Целевым значением являлся класс нагрузки на следующий час. Обучающая выборка содержала 28027 последовательностей, тестовая выборка — 7013 последовательностей.

В работе были построены и обучены три архитектуры рекуррентных нейронных сетей: SimpleRNN, LSTM и GRU. Для каждой архитектуры были протестированы четыре оптимизатора: SGD, Momentum, RMSProp и Adam, а также три варианта регуляризации: без регуляризации, L1 и L2. Всего было рассмотрено 36 конфигураций моделей.

Лучший результат по точности на тестовой выборке показала модель SimpleRNN с оптимизатором RMSProp и L1-регуляризацией. Её значение `test_accuracy` составило 0.9331. Это наибольшая точность среди всех рассмотренных моделей.

Лучшей LSTM-моделью стала конфигурация Adam + L2, показавшая `test_accuracy` = 0.9280 и `test_loss` = 0.2386. Лучшей GRU-моделью стала конфигурация Adam без регуляризации, показавшая `test_accuracy` = 0.9287 и `test_loss` = 0.1740. При этом именно GRU с Adam без регуляризации получила минимальное значение функции потерь среди всех моделей.

Худшие результаты в большинстве случаев показали конфигурации с обычным SGD. Особенно неудачным оказалось сочетание SGD и L1-регуляризации: для LSTM оно дало  $\text{test\_accuracy} = 0.8798$  и  $\text{test\_loss} = 1.0773$ , а для GRU —  $\text{test\_accuracy} = 0.8952$  и  $\text{test\_loss} = 0.9556$ . Это показывает, что для данной задачи адаптивные оптимизаторы Adam и RMSProp подходят лучше, чем обычный стохастический градиентный спуск.

По результатам экспериментов можно сделать вывод, что все три архитектуры способны решать поставленную задачу классификации нагрузки с высокой точностью. Однако наилучшей по accuracy оказалась SimpleRNN, а наилучшей по loss — GRU. Это показывает, что более сложная архитектура не всегда гарантирует максимальную точность, особенно если временное окно относительно небольшое и составляет 24 часа.

В результате выполнения лабораторной работы были закреплены практические навыки подготовки временных рядов, формирования последовательностей для рекуррентных нейронных сетей, построения моделей SimpleRNN, LSTM и GRU, применения различных оптимизаторов и регуляризации, а также анализа качества моделей по графикам accuracy/loss и итоговым метрикам на тестовой выборке.